

Lab: BoundedStack

แลปบูรณาการ: Clean Code · Automated Testing & TDD · Abstract Data Types

หัวข้อ	รายละเอียด
เนื้อหาที่บูรณาการ	Week 2 Clean Code & Defensive Programming, Week 3 Testing & TDD, Week 4 Abstract Data Types
รูปแบบ	งานเดี่ยว (ทำคนเดียว)
ขอบเขต	ADT เพียง 1 ตัว
ระยะเวลา	2 ชั่วโมงในคาบ + ส่งงานภายใน 7 วัน
ภาษา	Java

1. โจทย์ (Problem Statement)

โจทย์ของแลปนี้คือ:

จงออกแบบและพัฒนา ADT ชื่อ BoundedStack — สแต็กที่เก็บข้อมูลได้สูงสุดตามความจุที่กำหนดตอนสร้าง — ให้ผู้อื่นนำไปใช้ต่อได้จริงโดยไม่ต้องอ่านโค้ดภายใน และนิสิตต้องพิสูจน์ให้ได้ว่ามันถูกต้อง ด้วย test ที่รันได้ ไม่ใช่ด้วยคำพูด

นี่คือรายละเอียดทั้งหมดของโจทย์ ไม่มีการระบุ requirement รายข้อ ไม่มีรายการเมธอดที่ต้องมี ไม่มีตัวอย่าง input/output ให้ นิสิตต้องตัดสินใจเองทั้งหมดว่า:

- BoundedStack ควรทำอะไรได้บ้าง และอะไรที่ไม่ใช่หน้าที่ของมัน
- จะมี operation อะไรบ้าง ชื่ออะไร และแต่ละตัวมี spec อย่างไร (precondition / postcondition / กรณีผิดพลาด)
- จะเก็บ representation ด้วยโครงสร้างอะไร เพราะอะไรจึงเลือกแบบนั้น และจะซ่อนมันอย่างไร
- เมื่อสแต็กเต็มแล้วมีการใส่ข้อมูลเพิ่ม ควรเกิดอะไรขึ้น — โยน exception, คืนค่าบอกผลสำเร็จ, หรือไม่ทำอะไร ท่านเลือกทางไหนและเพราะอะไร
- เมื่อสแต็กว่างแล้วมีการหยิบข้อมูลออก ควรใช้ assertion หรือ exception จัดการ และเหตุผลคืออะไร
- ความจุเท่ากับศูนย์ หรือความจุติดลบ ถือว่าถูกต้องหรือไม่ และจะจัดการอย่างไร
- จะทำให้ ADT เป็น mutable หรือ immutable และผลของการเลือกนั้นคืออะไร
- จะทดสอบอย่างไรให้เชื่อได้ว่ามันถูกต้อง และคำว่า “พอ” หมายถึงเท่าไร
- จะจัดลำดับการทำงานอย่างไร (เขียนโค้ดก่อน หรือเขียน test ก่อน)

อาจารย์จะไม่ตอบคำถามว่า “ต้องทำอะไรบ้าง” แต่จะตอบคำถามว่า “ทำไมสิ่งที่คิดไว้ถึงดีหรือไม่ดี”

2. ข้อจำกัด (Constraints)

ข้อจำกัดต่อไปนี้เป็นกรอบเดียวที่กำหนดให้ ส่วนที่เหลือเป็นอิสระของนิสิต

- C1 — ส่ง ADT เพียง 1 ตัวคือ BoundedStack และ client ต้องไม่สามารถเข้าถึงหรือแก้ไข representation ได้โดยตรงไม่ว่าทางใด
- C2 — ต้องมี operation ครบทั้ง 4 บทบาท: Creator, Producer, Observer, Mutator อย่างน้อยบทบาทละ 1 ตัว (ถ้าเลือกทำ immutable type แล้วไม่มี Mutator ต้องอธิบายเหตุผลเชิงออกแบบให้ชัด)
- C3 — ต้องมี Abstraction Function และ Representation Invariant เขียนเป็น comment เหนือ private fields และต้องมี checkRep() ที่เรียกจริง
- C4 — ต้องเขียน test runner เอง ห้ามใช้ JUnit / TestNG / assertJ หรือ framework ใด ๆ ผลลัพธ์ต้องพิมพ์ PASS/FAIL และสรุปจำนวนรวมได้เอง
- C5 — ห้ามใช้ AI generate โค้ดหลักหรือ test ทั้งก่อน (ใช้ถามแนวคิดได้)
- C6 — โค้ดทั้งหมดต้องคอมไพล์และรันด้วย javac / java เปล่า ๆ ไม่มี build tool
- C7 — งานที่ส่งต้องมีหลักฐานลำดับการพัฒนา (commit log) ที่แสดงให้เห็นว่าท่านทำงานอย่างไร
- C8 — ขนาดงานที่คาดหวัง: ADT ประมาณ 80–150 บรรทัด และ test ประมาณ 20-40 – เคส — ถ้าเกินไปมาก แปลว่าขอบเขตกว้างเกินความจำเป็น
- C9 — ห้ามใช้ java.util.Stack หรือ Deque เป็น representation ภายใน (จะกลายเป็นการห่อของที่มีอยู่แล้ว ไม่ได้ออกแบบเอง) — ใช้ array หรือ ArrayList เป็นฐานได้

3. สิ่งที่ต้องส่ง (Deliverables)

นิสิตเป็นผู้ตัดสินใจว่าเนื้อหาในแต่ละชิ้นควรมีอะไร — ที่กำหนดคือ “ต้องมีชิ้นนี้” เท่านั้น

#	ชิ้นงาน	เกณฑ์ผ่านขั้นต่ำ
D1	BoundedStack.java	คอมไพล์ผ่าน · เป็นไปตาม C1–C3
D2	Test runner + test cases	รันได้ · รายงานผลได้เอง · เป็นไปตาม C4
D3	Design Document	spec ของทุก operation

4. เกณฑ์การประเมิน (Rubric)

เกณฑ์นี้บอกว่าอาจารย์ “มองหาอะไร” ไม่ได้บอกว่าท่าน “ต้องทำอะไร”

มิติ	คะแนน	อาจารย์มองหาอะไร
Abstraction	25	กำแพงนามธรรมแน่นหนาเพียงใด · เปลี่ยน representation ได้โดย client ไม่พึงหรือไม่
AF / RI / checkRep	15	AF และ RI เขียนได้ถูกต้อง แม่นยำ และ checkRep จับ rep ที่ผิดได้จริง
Test Quality	25	test แต่ละตัวมีเหตุผลรองรับ (partition / boundary / branch) ไม่ใช่สุ่มเตา
Clean Code	15	ข้อสื่อความหมาย · ฟังก์ชันทำงานเดียว · ไม่มี side effect ที่ซ่อนอยู่ · DRY
Defensive Programming	10	แยกใช้ assertion กับ exception ได้ถูกบริบท
Specification & Design Doc	10	spec ชัดจนคนอื่นเขียน test ได้โดยไม่ต้องเปิดโค้ด · อธิบายทางเลือกที่ตัดทิ้ง

5. คำถามที่ท่านต้องตอบให้ตัวเองก่อนส่งงาน

ไม่ต้องส่งคำตอบเหล่านี้ แต่ถ้าตอบไม่ได้ แปลว่างานยังไม่เสร็จ

- ถ้าพຼຽงนี้ต้องเปลี่ยน representation จาก array เป็น ArrayList (หรือกลับกัน) client จะพังก็บรืຽด?
- ถ้ามีคนแอบแก้โค้ดของท่านให้ผิดหนึ่งบรืຽด test ของท่านจะจับได้ไหม?
- test ตัวไหนที่ท่านเขียนขึ้นเพราะ “มันควรมี” มากกว่าเพราะ “มันเสี่ยงจริง”?
- ท่านทดสอบครบหรือยัง: ว้าง · มีตัวเดียว · เกือบเต็ม · เต็มพอดี · เต็มแล้วใส่เพิ่ม · ว้างแล้วหยิบออก · ความจุ 1 · ความจุ 0?
- ถ้ามีคนส่ง null / ค่าติดลบ / ค่าว่าง / ค่ามหาศาลเข้ามา จะเกิดอะไรขึ้น และท่านตั้งใจให้เกิดแบบนั้นหรือไม่?
- ถ้า client เอาค่าที่ได้จาก observer ไปแก้ สแตกของท่านจะเปลี่ยนตามไหม?
- ถ้าตัดข้อฟังค์ชันออกหมดแล้วให้คนนอกอ่าน เขาจะเดาได้ไหมว่าแต่ละตัวทำอะไร?

6. กำหนดการในคาบ (แนะนำ — ไม่บังคับ)

ช่วงเวลา	กิจกรรม
0:00–0:15	รับโจทย์ · ร่างรายการ operation และ spec ของตัวเอง
0:15–0:35	จับคู่แลกเปลี่ยน spec — ให้เพื่อนหาช่องโหว่ใน spec ของท่าน
0:35–1:00	ตัดสินใจเรื่อง representation · เขียน AF และ RI
1:00–2:00	ลงมือเขียน test และโค้ด (อาจารย์เดินให้คำปรึกษาแบบตั้งคำถาม)